REPORT: VehicleVision Classification

Logistic Regression, Engineered Tree Hybrid, and Soft Voting

Machine Learning Course Project | 15 October 2026

This report describes the code and saved outputs in **vehicle_logistic.ipynb**, **vehicle_dt.ipynb**, and **vehicle_ensemble.ipynb**. The three implementations predict vehicle classes from numerical specifications and write submission CSV files.

1. Problem and Dataset

The task is supervised classification into three categories: **0 - Sedan, 1 - SUV, and 2 - Sports Car**. The target is **vehicle_class**. The notebooks exclude **vehicle_id** from model inputs and preserve it in the submission.

Input file	Rows	Columns	Use
vehicle_train_10000.csv	10,000	22	Measurements and labels
vehicle_test.csv	2,500	21	Measurements for prediction

The original representation has 20 input features. The standalone LR and simple ensemble use those original features. The tree-hybrid notebook adds 17 derived features, producing 37 predictors.

2. Dataset Inspection

The LR notebook displays the training shape, the first five rows, the target classes, and class counts. These are the dataset-inspection results recorded in the supplied notebooks.

Vehicle class	Training rows
Sedan (0)	3,389
SUV (1)	3,326
Sports Car (2)	3,285

The three counts are similar. The simple ensemble also prints **7,500 cleaned training rows** after removing duplicate feature-and-label rows, and **2,500 test rows**. Its cleaning step therefore removes 2,500 repeated rows from the original training file.

Accuracy measures the fraction of correctly predicted labels. The LR and simple ensemble notebooks also print precision, recall, F1, and support through classification_report.

3. Preprocessing and Feature Engineering

3.1 Preprocessing used in each notebook

Implementation	Input preparation	Model preprocessing
Standalone LR	Exclude ID and target. Retain the 10,000 raw training rows.	Median imputation, followed by StandardScaler.
Tree hybrid	Create 17 features for both training and test data; exclude ID and target. Retain the raw training rows.	StandardScaler for its LR branch. No general imputer is included in this notebook.
Simple ensemble	Convert infinities to NaN, remove duplicate feature-and-label rows, and check the feature columns and target labels.	Separate median-imputation pipelines; add scaling only to LR.

Median imputation fills a missing numerical reading using a median learned from fitting rows. **StandardScaler** centers and scales features before LR. make_pipeline keeps these steps together with the classifier. The bagged trees in the simple ensemble use imputation without scaling.

3.2 Features added in vehicle_dt.ipynb

create_super_features() copies the input frame and adds the following feature families. The same function is applied to training and test measurements.

Family	Derived features	How they are used
Powertrain (6)	power_to_weight, torque_to_weight, battery_to_weight, battery_to_accel, power_per_cyl, torque_per_disp	Express power, torque, and battery readings relative to other specifications.
Geometry (4)	volume_est, density_est, wheelbase_to_length, length_to_width	Describe approximate body size, mass per estimated volume, and dimension ratios.
Aerodynamics (2)	frontal_area, drag_area	Combine width and height, then multiply by the drag coefficient.
Energy and handling (5)	kinetic_energy, braking_force, avg_fuel_efficiency, fuel_tank_to_weight, low_slung_ratio	Combine mass and braking distance, average fuel efficiency, and create capacity or clearance ratios.

For example, power_to_weight divides horsepower by weight. volume_est uses length x width x height with a unit conversion. kinetic_energy assumes 100 km/h, and braking_force divides that estimate by braking distance. These are derived numerical proxies, not additional measured attributes. Some ratios add 1e-5 to their denominator.

4. Models and Ensemble Methods

4.1 Logistic Regression

LR learns class scores from the supplied features and predicts a vehicle label. The standalone notebook and simple ensemble use **C=1.0, max_iter=3000, solver="lbfgs"**, with median imputation and scaling. C controls regularization strength, while max_iter caps optimization iterations.

4.2 Engineered tree hybrid in vehicle_dt.ipynb

Despite its filename, this notebook trains four tree ensembles and an LR model on all **10,000 rows and 37 features**. Its settings are:

Branch	Explicit settings
Gini bagging	300 trees; maximum depth 16; minimum leaf size 2; feature fraction 0.85; seed 42.
Entropy bagging	300 trees; entropy criterion; maximum depth 16; minimum leaf size 2; feature fraction 0.85; seed 101.
Random Forest	500 trees; minimum split size 4; seed 42.
Extra Trees	500 trees; minimum split size 4; seed 42.
LR branch	StandardScaler; C=1.0; maximum iterations 2,000; seed 42.

Bagging averages trees fitted to resampled data. **Random Forest** also samples features during split selection. **Extra Trees** adds random split thresholds. Gini and entropy are two split criteria. The tree branches contain **1,600 trees** in total.

4.3 Probability averaging in the two implementations

The engineered hybrid gives each tree branch one quarter of the tree probability average, then combines that average with its LR probabilities:

```
P_tree = (P_gini + P_entropy + P_RF + P_ET) / 4
P_hybrid = 0.40 * P_tree + 0.60 * P_LR
prediction = argmax(P_hybrid)
```

Thus each tree branch contributes 10% and LR contributes 60% to the final hybrid probabilities. The weights are assigned directly in the code.

The **simple ensemble** instead uses VotingClassifier(voting="soft") with **95% LR + 5% bagging**. Its bagging model has 80 trees, unrestricted maximum depth, minimum leaf size 5, sample fraction 0.8, feature fraction 1.0, bootstrap sampling, and seed 42. It uses the original 20 features, not the 37-feature hybrid.

Both approaches choose the class with the largest combined probability. The supplied notebooks do not run a weight search.

5. Validation Results and Discussion

5.1 How validation is performed

Standalone LR: train_test_split uses a 20% validation fraction, seed 42, and stratify=y. It fits on 8,000 raw rows and evaluates on 2,000. Stratification helps retain the class proportions in both splits.

Simple ensemble: identical measurement vectors form groups. StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=2026) supplies splits, but the code takes **only the first split**. It is a single grouped 80/20 validation check, not an average of five CV runs. The same validation rows are used for LR, bagging, and their ensemble; the weights remain fixed.

5.2 Saved accuracy results

Model / implementation	Accuracy	Evaluation recorded
Standalone LR	88.35%	2,000 raw validation rows.
LR inside simple ensemble	87.73%	Grouped split of cleaned data.
Bagging inside simple ensemble	87.13%	Same grouped validation rows.
95% LR + 5% bagging	87.67%	Same grouped validation rows.
37-feature engineered hybrid	Not reported	Fits all training rows and predicts test data; no validation step.

The 88.35% and 87.73% LR results use different training data, split procedures, and validation rows. They do not isolate the effect of cleaning. On the common cleaned split, LR is slightly stronger than bagging and the 95/5 blend; this run does not show an ensemble accuracy gain.

5.3 Printed class report for the simple ensemble

Vehicle class	Precision	Recall	F1	Support
Sedan	0.83	0.83	0.83	521
SUV	0.88	0.91	0.89	495
Sports Car	0.93	0.90	0.92	484

Values above use the notebook's displayed two-decimal precision. SUV has the highest displayed recall, and Sports Car has the highest precision. The class supports sum to 1,500 validation vehicles.

The hybrid notebook prints predicted test-class shares: SUV 33.36%, Sedan 33.32%, and Sports Car 33.32%. These describe its prediction distribution and do not measure correctness. No validation score is saved for that hybrid.

6. Model Choice and Submission Generation

6.1 Best supported result

Within the simple ensemble's common validation split, **LR gives the highest accuracy: 87.73%**, followed by the blend at 87.67% and bagging at 87.13%. LR is therefore the stronger baseline in that direct comparison. The engineered hybrid cannot be ranked against them from these notebooks because it has no recorded validation score.

The standalone LR notebook records 88.35% on its own split, but that does not establish superiority under the cleaned grouped protocol. The notebooks contain no test labels or recorded leaderboard scores, so this report makes no claim about hidden-test performance.

6.2 Final training and output files

Implementation	Final fitting data	Submission file
Standalone LR	All 10,000 raw rows; 20 features.	vehicle_submission.csv
Engineered hybrid	All 10,000 raw rows; 37 features for all five branches.	submission_hybrid_super_blend.csv
Simple ensemble	All 7,500 cleaned rows; 20 features and fixed 95/5 weights.	submission_simple_lr_bagging.csv

Every submission contains **vehicle_id** and an integer **vehicle_class**. The test file has 2,500 rows. ID values come from the test data, and predictions are generated in the same row order. The standalone LR refits its pipeline after validation and uses Google Colab's files.download() to download its saved CSV.

The engineered hybrid fits its models directly on the full training data, calculates the combined probability vector, and saves the class with the largest probability. The simple ensemble refits VotingClassifier on all cleaned rows after validation, then writes its predictions.

6.3 Checks implemented in the simple ensemble

Assertions check that test feature columns match training inputs, target labels are nonmissing and belong to {0, 1, 2}, and training/validation measurement groups do not overlap. Before saving, the notebook checks the submission row count, unique IDs, absence of missing values, valid predicted classes, and original test-ID order.

The other two notebooks display a submission preview or predicted class counts. They do not contain the same explicit submission assertions, so those stronger checks are attributed only to the simple ensemble.

6.4 Conclusion

The three notebooks implement a regularized LR baseline, a feature-engineered hybrid of four tree ensembles and LR, and a simpler fixed-weight LR/bagging vote. Their saved outputs support the reported local validation scores and successful CSV generation. The simple blend does not improve on its LR component in the recorded comparison; the larger hybrid needs its own validation before a performance advantage can be claimed.